SoundScript

The Complete Developer Guide — everything about how the project is built, why, and how to explain it.

Turn any document into a narrated audiobook — with chapters, read-along highlighting, and a private podcast feed.

Live demo: soundscript-web.onrender.com · Stack: Node 22 · Express · React 19 · Supabase · ElevenLabs · Render

A teaching reference for the developer — architecture, pipeline, deployment, and interview preparation.

Contents

1. What SoundScript is
2. The core mental model
3. Every feature, explained
4. Navigating the website
5. Navigating the app (views)
6. How it works: the pipeline
7. Request lifecycle, end to end
8. Architecture
9. Data model & the queue
10. The API surface
11. Tech stack & why each choice
12. Auth, security & RLS
13. Caching, quotas & cost control
14. The TTS provider abstraction
15. Frontend architecture
16. Testing & CI
17. Deployment (Render)
18. Design decisions to be able to defend
19. Known limits & future work
20. Interview Q&A — this project
21. Interview Q&A — general tech
22. One-minute cheat sheet

1 · What SoundScript is

A document-to-audiobook platform.

A user uploads a **PDF, EPUB, DOCX, Markdown, or plain-text file** (or pastes a short passage) and gets back a single MP3 narrated in a natural voice — with pauses in the right places, clickable chapters, read-along highlighting that tracks the spoken sentence, cross-device resume, and an optional private podcast feed they can subscribe to in any podcast app.

The single most important thing to understand — and the line that frames every design decision — is this:

> **The text-to-speech API call is one line in the middle of the system. Everything that makes the result usable happens on either side of it.** A raw PDF has no paragraphs, repeats its running header on every page, and hyphenates words across line breaks. TTS APIs cap input length and bill per character. Synthesis is slow. So SoundScript is really a **document-processing and job-orchestration pipeline that happens to end in audio** — not a thin wrapper around an API.

Framing it this way in an interview immediately signals that you understand where the real engineering is: format detection, text cleaning, chunking, a concurrent job queue, caching, and audio stitching — not the `synthesize()` call itself.

2 · The core mental model

Hold three ideas and the whole codebase falls into place:

1. **It is a pipeline.** Text flows through fixed stages: *extract* → *clean* → *apply pronunciation rules* → *chunk* → *queue* → *synthesize (workers)* → *stitch* → *audiobook.* Each stage is one module with one job.
2. **The database is the queue.** There is no Redis, no RabbitMQ. The `chunks` table *is* the work queue, and Postgres row locking (`FOR UPDATE SKIP LOCKED`) is what lets many workers pull from it safely.
3. **Identity is enforced by the database, not the app.** Every request carries a Supabase JWT; the user id comes from the verified token, and Postgres Row-Level Security (RLS) guarantees a user can only ever touch their own rows — even if the API code had a bug.

3 · Every feature, explained

Feature	What it does & why it matters
Multi-format ingest	Accepts PDF, EPUB, DOCX, TXT, Markdown. Format is detected from the file's *own bytes* (magic numbers), never the extension or the client-supplied MIME type — both of which are trivially spoofed.
Scanned-PDF OCR	If a PDF has no text layer, it falls back to Tesseract OCR (`tesseract.js`). Best-effort, English by default; the ~10 MB model downloads on first use.
Smart cleaning	For PDFs: strips running headers/footers (found by cross-page repetition), removes page numbers, de-hyphenates words split across lines, and rebuilds paragraphs the PDF never actually had.
Cost estimate first	Before committing, the app quotes chunk count, cache hits, and *billable* characters, and refuses if it would blow your daily quota. The estimate and the real upload run the same parsing code, so the quoted price can never drift from the charged price.
Voice picker + preview	Choose a provider (ElevenLabs / OpenAI) and voice, and hear a short sample before committing to a whole document.
Chaptered audiobook	Measured pauses — 0.35 s between sentences, 0.75 s between paragraphs — and every paragraph becomes a clickable chapter with a real timestamp.
Read-along	The sentence currently being spoken is highlighted in step with playback; click any sentence to jump the audio there.
Real player	Variable playback speed (0.75×–2×), ±15 s skip, and cross-device resume (your position is saved server-side).
Pronunciation rules	Per-user find→replace pairs (e.g. `"RLS" → "R L S"`) applied *before* synthesis so initialisms and names are spoken correctly.
Private podcast feed	Subscribe to your own library as an RSS feed in any podcast app, via a secret, rotatable token URL.
Live progress	An animated "pipeline theater" driven by *real* server-sent events (SSE) — every number is genuine, not a fake spinner.
Guests + accounts	Use it instantly as an anonymous guest; attach an email later and keep all your history on the same account.

Light / dark themes	A "Mahogany Library" palette with a persisted theme toggle; fully responsive with a mobile drawer nav.

4 · Navigating the website

The public product lives at two URLs (both on Render's free tier):

- **Web app** — `https://soundscript-web.onrender.com` (the React single-page app the user interacts with).
- **API** — `https://soundscript-api-…onrender.com` (the Node/Express backend; you rarely visit it directly except `/api/health`).

> **Cold-start note.** On the free plan the API sleeps after inactivity, so the very first request of the day can take ~30–50 seconds to wake the server. This is normal free-tier behaviour, not a bug — it's worth mentioning in an interview as a known trade-off you chose deliberately for a zero-cost demo.

First visit flow: open the web URL → you are signed in *anonymously and automatically* (no login wall) → you land on the home screen with a text box. From there you either paste a short passage and convert it directly, or click *"Turn it into an audiobook"* to open the Library and upload a full document.

5 · Navigating the app (the views)

The frontend is a single-page app with no client-side router — navigation is state-driven inside `App.jsx`, which conditionally renders one "view" at a time. The persistent chrome is a top nav bar (desktop) / hamburger drawer (mobile) plus a floating theme toggle visible everywhere. The destinations are:

View	Component	What you do there
Home	`App.jsx`	Paste up to 500 characters of text and convert it to speech instantly. Shows an inline audio player + download link. This is the "quick" path.
Library	`Library.jsx`	The document workflow. Upload a PDF/EPUB/DOCX/TXT, pick a voice (with preview), see the *cost estimate*, confirm, then watch it process. Lists all your documents; open or delete them.
Document view	`DocumentView.jsx`	The audiobook player: play/pause, speed control, ±15 s skip, chapter list, and read-along text that highlights the current sentence and lets you click to jump. Also where a failed job can be retried.
Pipeline theater	`PipelineTheater.jsx`	The live progress screen shown while a document synthesizes — stages light up from real SSE events; per-chunk cells fill in as each chunk finishes.
History	`History.jsx`	Your past single-shot text conversions (read directly from Supabase under RLS).
Settings	`Settings.jsx`	Manage pronunciation rules (add/delete find→replace pairs) and your private podcast feed URL (view / copy / rotate the token).
Auth	`Auth.jsx`	"Save my work" — attach an email + password to your anonymous account (keeps the same user id and all history), or sign into an existing account.

Guest → account nuance worth knowing: upgrading a guest calls `supabase.auth.updateUser()` (attaches credentials to the *existing* anonymous uid, so history is preserved). Calling `signUp()` instead would mint a brand-new uid and silently orphan everything the guest had done — a subtle bug the code deliberately avoids.

6 · How it works: the pipeline

```
file ──► extract ──► clean ──► rules ──► chunk ──► queue ──► workers ──► stitch ──► audiobo
       (per format) (pdf only)(per user)  (fit cap) (postgres)  (N par.)   (ffmpeg)
```

Stage	Module	What it does
Extract	`lib/extract/`	Detects format from magic bytes, then extracts text: PDF via pdfjs (paragraphs inferred from baseline gaps), EPUB via spine walk, DOCX via paragraph runs, TXT/MD directly, or OCR fallback for scanned PDFs.
Clean	`lib/text/clean.js`	PDF-only heuristics: drop headers/footers found by repetition across ≥ 50% of pages, strip page-number lines, de-hyphenate, rejoin wrapped lines into real paragraphs.
Rules	`lib/text/pronounce.js`	Applies the user's find→replace rules *before* hashing, so corrected text caches under its own key. Longest pattern first; each rule runs once (no cascading).
Chunk	`lib/text/chunker.js`	Splits text to fit the API's character cap, preferring boundaries in order: paragraph → sentence → clause → word, only descending when the level above won't fit. Keeps paragraph position so stitching knows where the long pauses go.
Queue	`supabase/migrations`	The `chunks` table *is* the queue. `claim_next_chunk()` uses `FOR UPDATE SKIP LOCKED` so concurrent workers never grab the same row twice.
Workers	`lib/worker/pool.js`	N workers pull from the shared queue, synthesize (with retry + jittered backoff), write audio to storage, and populate the cache.
Stitch	`lib/audio/stitch.js`	Concatenates chunk audio with inserted silence (0.35 s / 0.75 s), re-encodes to one clean MP3 via ffmpeg, and emits chapter timestamps + a per-chunk timeline for read-along.

WHY SENTENCE DETECTION IS ITS OWN MODULE

Splitting on `/[.!?]\s/` is wrong in ways you hear instantly: "Dr. Smith" becomes two sentences with a pause after "Dr", and "3.14" becomes "3." and "14". `lib/text/sentences.js` carries an abbreviation set (titles, academic refs) and number handling so boundaries land where a human would put them.

WHY THE CHUNKER DESCENDS THROUGH BOUNDARY LEVELS

Where a split lands is *audible*. Break mid-sentence and the narration hits a wall and restarts. So the chunker tries to keep whole paragraphs together; if a paragraph exceeds the cap it splits at sentences; if a sentence is still too big it splits at clauses (after commas/semicolons); and only for a pathological single "word" longer than the whole budget (a URL or hash) does it hard-split inside a token.

WHY STITCHING RE-ENCODES INSTEAD OF COPYING BYTES

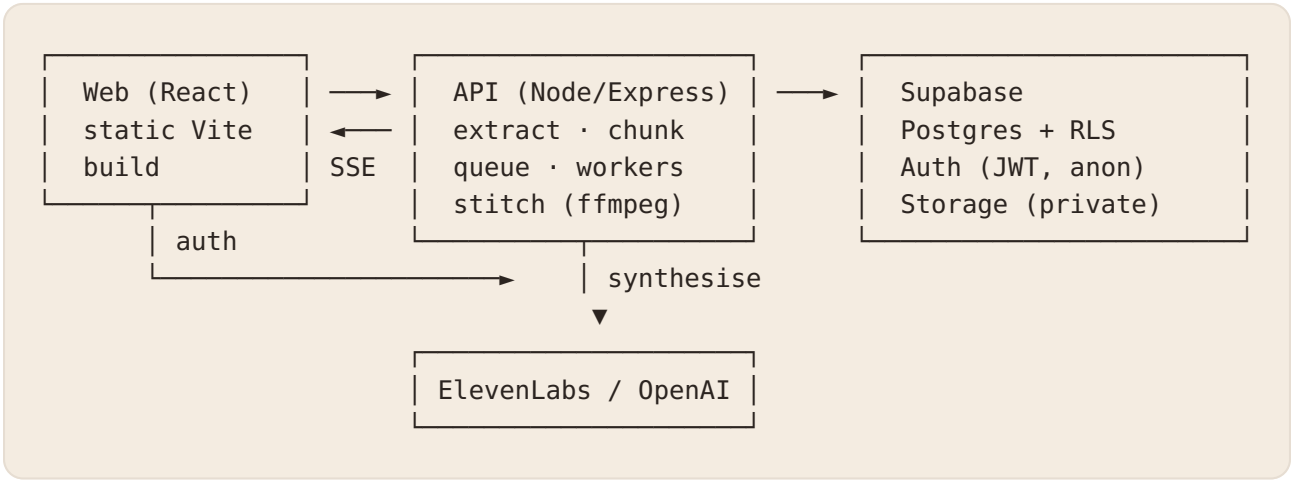
The ffmpeg `concat` demuxer requires every input to share codec parameters. A raw `-c copy` would paste mismatched frames together into a file that *plays wrong rather than failing loudly* the moment a different voice, model, or an older cache entry is mixed in. Re-encoding normalizes everything to 128 k mono 44.1 kHz — costing seconds against minutes of synthesis. Chapter timestamps are also measured from the *real* encoded lengths (silence included), because guessing from character counts drifts within a page.

7 · Request lifecycle, end to end

Follow a document upload from click to playback:

1. **Browser → API.** `Library.jsx` POSTs the file (multipart) to `/api/documents` with a `Bearer` JWT. First it POSTs with `estimate=1` to get the cost quote; on confirm it POSTs for real.
2. **Auth.** `requireAuth` verifies the JWT with Supabase, sets `req.user` from the *verified token*, and builds a user-scoped Supabase client so every DB call runs under that user's RLS.
3. **Parse.** The buffer goes to `lib/extract` (format from bytes) → clean → pronunciation rules → chunk. The estimate and real path share this exact code.
4. **Quota check.** `lib/quota.js` sums billable characters in the last 24 h; if the new job would exceed `DAILY_CHAR_QUOTA`, it's refused before any API call.
5. **Persist.** A `documents` row is created, then `chunks` rows are inserted (each with its content hash, paragraph index, and "ends paragraph" flag). Cache hits are marked immediately. The original file is stored in the private bucket only after everything else succeeds.
6. **Queue a job.** A `jobs` row is inserted as `queued`. In the default in-process mode the API starts the worker pool right away (after responding, so a slow first chunk can't delay the HTTP response); with `INLINE_PROCESSING=false` the standalone `worker.js` picks it up instead.
7. **Synthesize.** `processDocument` atomically claims the job, reads the document's voice once, and runs N workers. Each pulls a chunk via `claim_next_chunk`, checks the cache, synthesizes with retry, uploads the MP3, writes the cache, marks the chunk done, and `publish()`es an SSE event.
8. **Stream progress.** The browser holds open `GET /api/documents/:id/events` (SSE over `fetch`, so it can send the auth header). `PipelineTheater.jsx` renders each event live.
9. **Stitch.** Once every chunk is done, `assembleDocument` downloads the pieces and `stitchChunks` produces the final MP3 + chapters + timeline, stored as an `audio_assets` row.
10. **Playback.** `DocumentView.jsx` requests a short-lived *signed URL* for the private audio object and plays it, syncing highlight and resume position against the timeline.

8 · Architecture



Two deployable pieces, plus Supabase as the backing platform:

- **Web** — a static Vite build. Talks to Supabase *directly* for auth, and to the API for everything else. No server of its own; it's just files on a CDN.
- **API** — a *persistent* Node process (crucially **not serverless**). A 40-page chapter takes minutes to synthesize; the API returns a job id instantly and keeps working in the background while streaming progress over long-lived SSE connections.
- **Supabase** — Postgres (holds the work queue and enforces RLS), Auth (JWT + anonymous sign-in), and Storage (a private bucket for audio, served via short-lived signed URLs).

> **Why not serverless?** This is a favourite interview question and the `render.yaml` comments call it out explicitly. Serverless functions are killed once the HTTP response is sent — which would end every synthesis job at the moment it started, and would sever every open SSE stream. A persistent process keeps the worker pool *and* the event stream alive for the whole job. That single constraint is the reason the deploy target is a persistent service.

9 · Data model & the queue

All state lives in Postgres (via Supabase migrations). The important tables:

Table	Purpose
`documents`	One row per uploaded document: title, source type, and the chosen voice (`provider` , `voice_id` , `model_id`). The document row owns the narrator.
`chunks`	**This is the queue.** One row per text chunk: text, `content_hash` , `idx` , `paragraph_idx` , `status` (pending/processing/done/failed), `attempts` , `from_cache` , and the audio `path` .
`jobs`	One row per synthesis run of a document: `queued → running → succeeded/failed` , with timestamps used by the stale-work reaper.
`audio_cache`	Content-addressed cache: `(user_id, content_hash) → storage path` . A cache hit means zero API cost.
`audio_assets`	The finished stitched audiobook: path, duration, byte size, per document.
`tts_conversions`	Single-shot text conversions (the Home path + History view).
`listening_positions`	Saved playback position per document, enabling cross-device resume.
`pronunciation_rules`	Per-user find→replace pairs.
`user_feeds`	The secret token for each user's private podcast feed URL.

THE TWO KEY STORED PROCEDURES (RPCS)

`claim_next_chunk(document_id)` — selects the next pending chunk `FOR UPDATE SKIP LOCKED` and flips it to `processing` atomically. *Skip locked* is the whole trick: two workers asking at the same instant get *different* rows rather than blocking or double-claiming.

`document_progress(document_id)` — counts chunk rows by status. Progress is **derived, not counted**: it queries the rows rather than incrementing a column, because a counter and the rows it summarizes drift apart the first time something fails or retries.

10 · The API surface

Every route below `/api` requires a Bearer JWT (via `requireAuth`). The public podcast feed is the one exception — it authenticates with the token embedded in its URL, because podcast apps can't send auth headers.

Method & path	Purpose
`GET /api/health`	Reports which keys/features are configured (elevenlabs, openai, supabase, ocr, feeds, inlineProcessing). Lets you diagnose a bad deploy without reading logs.
`POST /api/convert`	Single-shot synthesis of ≤ 500 chars. Rate-limited per IP.
`POST /api/documents`	Upload a document (multipart). With `estimate=1` returns a cost quote and writes nothing; otherwise creates the document, chunks, and queues a job.
`GET /api/documents` · `GET /api/documents/:id`	List documents / fetch one with its status & asset.
`GET /api/documents/:id/text`	Chunk text in reading order, for the read-along view.
`GET /api/documents/:id/events`	SSE stream of live progress for a job.
`PUT /api/documents/:id/position`	Save playback position (cross-device resume).
`POST /api/documents/:id/process`	Requeue / retry a document (the recovery path after a crash).
`DELETE /api/documents/:id`	Delete a document and its objects.
`GET /api/voices` · `POST /api/voices/preview`	List providers/voices; return a short spoken sample.
`GET/POST/DELETE /api/pronunciations`	Manage pronunciation rules.
`GET /api/feed` · `POST /api/feed/rotate`	Get the private feed URL; rotate its token (revokes the old one).
`GET /feeds/:token`	*Public* RSS endpoint for podcast apps.

11 · Tech stack & why each choice

Layer	Choice	Why this, and not the alternative
Backend	Node.js 22+, Express (CommonJS)	The whole job is I/O — HTTP to TTS providers, DB calls, file streaming — which is exactly what Node's event loop is best at. Express is minimal and well-understood. Node 22 specifically because `@supabase/supabase-js` v2.86 needs a native WebSocket that Node only ships from 22 onward (on 20 it throws at startup).
Frontend	React 19, Vite, Tailwind	Vite gives instant dev startup and a tiny static production bundle (no server needed). Tailwind keeps styling co-located with markup so the whole SPA stays a handful of files. React's state model fits the "one view at a time" navigation without a router.
DB / Auth / Storage	Supabase (Postgres + RLS + Auth + Storage)	One platform gives a real relational DB, JWT auth (including anonymous sign-in), and object storage — and RLS pushes authorization *into the database*, so security doesn't depend on the app remembering to filter every query. Fewer moving parts than stitching together Auth0 + S3 + a managed Postgres.
Text-to-speech	ElevenLabs (primary), OpenAI (optional)	ElevenLabs has the most natural long-form voices; OpenAI is a cheaper fallback and a second provider so a single vendor's quota can't take the app down. Both sit behind one abstraction so adding a third is one file.
Audio	ffmpeg / ffprobe (static binaries)	The de-facto standard for audio; static builds mean no system install on the host. Used to generate silence, measure real durations, and stitch/normalize the final MP3.
Parsing	pdfjs-dist + @napi-rs/canvas, adm-zip + fast-xml-parser, tesseract.js	pdfjs is Mozilla's battle-tested PDF engine (needs canvas for `DOMMatrix`); EPUB/DOCX are just zips of XML, so adm-zip + a fast XML parser is enough; tesseract.js is pure-JS OCR so there's nothing extra to install.
Deploy / CI	Render (Blueprint) + GitHub Actions	Render runs a *persistent* Node service (required, see architecture) and a static site from one `render.yaml`. GitHub Actions runs tests + lint + format on every push.

Tests / quality	node:test (100 tests), ESLint, Prettier	The built-in test runner means zero test-framework dependencies. Fixtures (PDF/EPUB/DOCX) are generated in memory as reviewable code, not committed as opaque binaries.

> **A note on "Vercel."** You may see a `app.set("trust proxy", 1)` line commented "Behind Vercel/proxies." The *actual* production deployment is **Render**, not Vercel — configured entirely by `render.yaml`. The `trust proxy` setting matters on *any* platform that puts a reverse proxy in front of the app (Render included): without it, `express-rate-limit` sees the proxy's IP instead of the real client IP and would rate-limit everyone as one. If asked "where is it deployed," the honest answer is Render; the proxy setting is platform-agnostic.

12 · Auth, security & RLS

- **Identity comes from the verified JWT, never the request body.** The API originally trusted a `user_id` in the body — which let any caller act as any user by typing a different UUID. Now the id is read only from the token Supabase verifies, and the token is forwarded so `auth.uid()` resolves inside Postgres and RLS applies as written.
- **RLS is the real security boundary.** Every table has policies of the form `auth.uid() = user_id` for select/insert/update/delete. Even if an API route forgot a `WHERE user_id = …`, the database still refuses to return another user's rows. Security doesn't depend on the app being bug-free.
- **Guests are real users.** Anonymous sign-in gives every visitor a genuine UUID, so history, quotas, and RLS all work through *one* code path — no `user_id = "guest"` special case to get wrong.
- **CORS + Bearer, not cookies.** Auth is a Bearer JWT, not a cookie, so a hostile origin can't forge a session via CSRF. `ALLOWED_ORIGINS` is an allow-list; `*` is safe here precisely because RLS guards every row regardless of who asks.
- **Private storage + signed URLs.** Audio lives in a private bucket; playback uses short-lived signed URLs, so links can't be shared or guessed permanently. The podcast feed uses longer-lived signed enclosures (podcast apps fetch hours later) plus a rotatable feed token.
- **The anon key is safe in the browser.** It's meant to be public; RLS is what protects data, not secrecy of that key. The *service-role* key (which bypasses RLS) lives only on the server and enables the feed + standalone worker.

13 · Caching, quotas & cost control

Audio is content-addressed. The cache key is `sha256(provider:voice:model:text)`. Re-uploading an unchanged document, or a repeated heading within one document, costs nothing and returns instantly. Voice and model are part of the key because the same text in a different voice is genuinely different audio.

Cache-before-mark ordering. A worker writes the `audio_cache` row *before* marking the chunk done. If the process dies between the two, a retry finds the cache entry and skips the paid API call — worst case a wasted upload, never a wasted (billed) synthesis.

Quota is measured from actual usage. `lib/quota.js` sums real billable characters in a rolling 24 h window across both paths (single-shot conversions + non-cached chunks). Because the estimate and the real upload share the same parsing code, the

quoted cost can't drift from the charged cost. Default cap: 10,000 chars/day (matching the ElevenLabs free tier's monthly ceiling).

14 · The TTS provider abstraction

`lib/tts/index.js` is the one interface every caller uses. Providers (`elevenlabs.js`, `openai.js`) each only know how to translate *one* request and classify *one* error; everything shared — defaults, dispatch, the retry policy, and the cache-key format — lives in the index. So adding a provider is one file plus one registry entry.

Retry with jittered exponential backoff. `withRetry` retries only *retryable* errors (429, 5xx) and gives up immediately on permanent ones (401, 422) rather than burning the budget proving they'll fail. The backoff is *jittered* because the pool runs several calls at once: without jitter a 429 hits every worker simultaneously, they all sleep the same 1000 ms, and they all retry in the same instant — rebuilding the exact spike that caused the limit. It also honours a server's `Retry-After` when present.

Fail-fast on fatal errors. If a worker hits a 401 (bad key) or 429 (quota exhausted), the whole pool stops — turning a hundred pointless calls into one clear failure message.

15 · Frontend architecture

- **State-driven navigation, no router.** `App.jsx` holds boolean/id state (`showLibrary`, `openDocumentId`, …) and renders exactly one view. Nav links are defined once and shared between the desktop bar and the mobile drawer so the two can't drift.
- **Direct-to-Supabase for auth & simple reads.** The client talks to Supabase directly for session/auth and for the History view; everything heavy goes through the API. `api.js` centralizes every backend call and always attaches a fresh Bearer token.
- **SSE over `fetch`, not `EventSource`.** The native `EventSource` can't send an `Authorization` header, and the usual workaround (JWT in the query string) leaks the token into every proxy log. So the client reads the stream manually via `fetch` + a reader, parsing `data:` frames itself.
- **Read-along keys on the timeline.** The stitched output ships a per-chunk timeline (start/end seconds); the sentence under the playhead is the text being spoken, which is what drives the highlight and click-to-jump.
- **Every progress number is real.** The "pipeline theater" is fed by genuine SSE events — no fake spinners — which is both a nice UX touch and an honest reflection of the backend.

16 · Testing & CI

100 unit tests run on Node's built-in `node:test` — no test framework dependency, and `npm test` needs no credentials. Coverage spans the chunker, sentence boundaries, PDF cleaning, EPUB/DOCX extraction, format detection, pronunciation rules, retry/backoff, ffmpeg stitching (with *real* audio), RSS rendering, and error classification. PDF/EPUB/DOCX fixtures are generated in memory as reviewable code rather than committed as opaque binaries — so a reviewer can see exactly what's being tested. GitHub Actions runs tests, ESLint, and Prettier in parallel on every push to `main`.

17 · Deployment (Render)

One `render.yaml` Blueprint deploys **both** services from a single repo: the persistent Node API and the static Vite web build. Push to `main` → Render auto-redeploys both; GitHub Actions runs the checks in parallel.

THE BUILD/RUN CONFIGURATION, AND WHY

- **API uses `npm ci` (not `install`)** so it installs exactly what `package-lock.json` pins — a deploy can't silently pick a different version than the one tested.
- **Full install, no `--omit=optional`.** pdfjs needs `@napi-rs/canvas` for `DOMMatrix` on every PDF, and that package ships its native binary as an *optional* platform dependency; `--omit=optional` would strip it and break all PDF parsing. The full install also brings in `tesseract.js`, so scanned-PDF OCR works (its ~10 MB model downloads on first use, not at build time).
- **Health check at `/api/health`** so Render restarts the service if it stops answering, and a misconfigured deploy is visible without reading logs.
- **Web is a static publish** of `tts-frontend/dist` with a catch-all rewrite to `index.html` so a hard refresh on any path still serves the app.
- **Secrets via `sync: false`** — set in the Render dashboard, never in the repo. Cross-references (`ALLOWED_ORIGINS`, `VITE_BACKEND_URL`) are hard-coded to the default service URLs and only need changing if a service is renamed.

FIRST-TIME DEPLOY CHECKLIST

1. Push the repo to GitHub.
2. Point the Supabase CLI at the prod project: `supabase link` → `supabase db push` (tables, RLS, RPCs, buckets) → `supabase config push` (enables anonymous sign-in — the guest flow depends on it).
3. Render → New → Blueprint → pick the repo; Render reads `render.yaml` and prompts for each `sync: false` secret (Supabase URL + anon key, ElevenLabs key, optionally OpenAI + service-role key).
4. Verify: `/api/health` returns `{"ok":true,…}` with the right features enabled; upload a small doc and confirm it narrates end to end.

> **Durability trade-off.** By default the API synthesizes *in-process*, so work in flight is lost if the free instance sleeps or restarts mid-job — but `POST /api/documents/:id/process` requeues it, and cached chunks are skipped so a retry is cheap. For real resilience you run `worker.js` as a *separate* Render service with `INLINE_PROCESSING=false` and the service-role key; it has a stale-work reaper that returns crashed chunks (stuck in `processing` > 10 min) to the queue on boot and periodically.

18 · Design decisions to be able to defend

> **Identity from the verified JWT, never the body.** Forwarding the caller's token means `auth.uid()` resolves and RLS applies exactly as written.

> **Guests are real users.** One anonymous-UUID code path for history, quota, and RLS — no special-casing.

> **Audio is content-addressed.** `sha256(provider:voice:model:text)` makes re-uploads free and instant.

> **The queue is the `chunks` table.** `FOR UPDATE SKIP LOCKED` makes concurrent workers safe with no extra infrastructure.

> **Progress is derived, not counted.** Counting rows can't drift from reality the way an incremented counter does after a failure.

> **The worker pool takes a client rather than making one.** A user-scoped client runs under RLS; a service-role client drains every queue from a standalone process. Only the trigger changes.

> **Estimate and real upload share one code path.** The confirm-screen cost can never diverge from what you're actually charged.

19 · Known limits & future work

- **In-process synthesis** is lost if the server restarts mid-job (the retry endpoint requeues it); the standalone worker with the reaper is the durable option.
- **SSE progress is per-process.** Multiple instances behind a load balancer would need a shared channel — Postgres `LISTEN/NOTIFY` or Supabase Realtime — because a client attached to instance A hears nothing about a job on instance B.
- **The cache is per-user.** A global cache would save more but would leak whether someone else had synthesized the same text.
- **OCR is best-effort** (clean scans, English by default).
- **ElevenLabs' free tier is 10,000 chars/month** — a real chapter is ~100k characters; OpenAI or a paid tier is the way around it.

20 · Interview Q&A — this project

Q. Give me the 30-second pitch.

SoundScript turns documents into narrated audiobooks. You upload a PDF, EPUB, DOCX or text file and get back a single MP3 with natural pauses, clickable chapters, and read-along highlighting, plus a private podcast feed. The interesting engineering isn't the text-to-speech call — that's one line — it's everything around it: detecting and cleaning messy PDFs, chunking text so splits aren't audible, a Postgres-backed concurrent job queue, content-addressed caching to avoid paying twice, and ffmpeg stitching with real chapter timestamps.

Q. Why is the backend a persistent server and not serverless?

Two reasons, both fatal to serverless. First, synthesizing a long document takes minutes; a serverless function is killed once it returns its HTTP response, which would end the job the instant it started. Second, the app streams live progress over long-lived SSE connections, and serverless can't hold a connection open past the response. A persistent process keeps both the worker pool and the event stream alive for the whole job.

Q. How does the job queue work without Redis or a message broker?

The `chunks` table *is* the queue. A stored procedure, `claim_next_chunk`, does `SELECT … FOR UPDATE SKIP LOCKED` to grab the next pending row and flip it to `processing` atomically. `SKIP LOCKED` is the key: if two workers ask simultaneously, one locks the row and the other skips past it to the next one, so no chunk is ever claimed twice and workers never block each other. It's a real queue with exactly-once claiming, using only Postgres.

Q. How do you make concurrent workers safe?

Three layers. The chunk claim uses `FOR UPDATE SKIP LOCKED`. The *job* claim is an `UPDATE … WHERE status='queued'` that returns a row only to the caller who won, so a double-trigger can't run two pools over one document. And chunks carry an `attempts` counter with a max, so a chunk that keeps killing workers eventually fails instead of looping forever.

Q. What happens if the server crashes mid-job?

In the default in-process mode, in-flight work stops — but nothing is corrupted, because audio is cached *before* the chunk is marked done, so a retry skips the paid API call. `POST /api/documents/:id/process` requeues the document. In the durable mode, the standalone worker's reaper finds chunks stuck in `processing` longer than 10 minutes (their owner is clearly gone), flips them back to `pending`, and the `attempts` counter survives so they don't retry forever.

Q. Walk me through your caching. What's the key?

`sha256(provider:voice:model:text)`. Voice and model are in the key because the same text in a different voice is different audio. It's per-user and content-addressed, so re-uploading an unchanged document — or a repeated heading inside one document — is free and instant. I also write the cache row before marking the chunk done, so a crash between the two costs at most a wasted upload, never a wasted (billed) synthesis.

Q. How do you keep the cost estimate honest?

The estimate endpoint and the real upload run the *same* parsing, cleaning, and chunking code — the estimate just stops before writing anything. So the chunk count, cache hits, and billable characters on the confirm screen are exactly what you'll be charged. There's no separate "estimator" that could drift from reality.

Q. Why chunk text at all, and how do you choose split points?

TTS APIs cap input length, so a 40-page chapter can't go in one call. Where the split lands is audible — break mid-sentence and the narration restarts against a wall. So the chunker prefers boundaries in order: paragraph, then sentence, then clause (after

commas), then word, descending only when the level above won't fit the cap. It only ever splits inside a word for a pathological token longer than the whole budget, like a URL.

Q. How does read-along highlighting stay in sync?

Stitching measures each chunk's *real* encoded duration (including inserted silence) and emits a timeline of start/end seconds per chunk. The frontend finds the chunk under the current playhead and highlights it; clicking a sentence seeks to that chunk's start. Because the timeline comes from measured durations, not estimated-from-character-count, it doesn't drift over a long document.

Q. Why re-encode the audio instead of just concatenating the MP3 bytes?

ffmpeg's concat demuxer requires every input to share codec parameters. A byte copy would paste mismatched frames into a file that plays wrong rather than failing loudly the moment you mix voices, models, or older cache entries. Re-encoding to one canonical format (128k mono 44.1kHz) normalizes everything, and it costs seconds against minutes of synthesis.

Q. How is authentication and authorization handled?

Auth is a Supabase JWT sent as a Bearer token. The user id is taken only from the verified token, never from the request body — trusting the body would let anyone impersonate anyone by changing a UUID. Authorization is enforced by Postgres Row-Level Security: every table has `auth.uid() = user_id` policies, so even a buggy query can't return another user's data. The app forwards the token so `auth.uid()` resolves inside the database.

Q. Guests can use it with no signup — how does that not break your data model?

Anonymous sign-in gives every guest a real UUID, so they flow through the exact same RLS, history, and quota code as a signed-up user. When they later "save their work," I call `updateUser` to attach an email to that *same* uid, so their history carries over. Using `signUp` would create a new uid and orphan everything — a bug I specifically avoided.

Q. Why SSE instead of WebSockets or polling?

Progress is one-directional server→client, so SSE is the right-sized tool — simpler than WebSockets, and it pushes the instant a chunk finishes instead of polling on a timer. I implement it over `fetch` rather than the native `EventSource`, because `EventSource` can't send an auth header and putting the JWT in the query string would leak it into proxy logs.

Q. How do you handle rate limits and transient failures from the TTS provider?

Retryable errors (429, 5xx) get exponential backoff with jitter; permanent ones (401, 422) fail immediately instead of wasting attempts. Jitter matters because several workers run at once — without it a 429 hits them all, they sleep the same interval, and they retry in the same instant, rebuilding the spike. I also honour `Retry-After`, and if a worker sees a 401 or quota-exhausted 429 the whole pool stops, turning a hundred doomed calls into one clear error.

Q. Why Supabase over rolling your own?

It gives me a real Postgres database, JWT auth with anonymous sign-in, and object storage on one platform — and crucially RLS, which moves authorization into the database so security isn't dependent on the app remembering to filter every query. Rolling my own would mean stitching together an auth provider, S3, and managed Postgres, and re-implementing row-level security in application code.

Q. What are the biggest limitations, and how would you scale it?

SSE is per-process, so horizontal scaling needs a shared channel — Postgres `LISTEN/NOTIFY` or Supabase Realtime. In-process synthesis loses in-flight work on restart, so I'd move to the standalone worker as the default. The cache is per-user for privacy; a global cache would save more compute. And the free ElevenLabs tier caps at 10k chars/month, so production means a paid tier or leaning on the OpenAI fallback.

Q. How did you detect file formats safely?

From the file's own bytes, never the extension or MIME type — both are client-supplied and spoofable. PDF declares itself in its first four bytes (`%PDF`). EPUB and DOCX are both zips, so I break the tie by scanning for a marker path inside: `word/document.xml` means DOCX, `META-INF/container.xml` means EPUB. Anything else falls through to plain text.

Q. What was the hardest part?

Making messy PDFs sound like prose. A PDF has no concept of a paragraph — extraction gives one line per visual line, the running header repeats on every page, page numbers sit on their own lines, and words are hyphenated across line breaks. The cleaning stage is a stack of heuristics: find furniture by cross-page repetition, strip page numbers, de-hyphenate, and rejoin wrapped lines. Every rule exists because of a specific way naive extraction sounded wrong when read aloud.

21 · Interview Q&A — general tech you should know

Q. What is Row-Level Security and why does it matter?

RLS is a Postgres feature that attaches authorization policies to a table so the *database* filters rows by the current user, not the application. A policy like `USING (auth.uid() = user_id)` means a query only ever sees that user's rows, even if the app code forgets a `WHERE` clause. It's defence in depth: authorization lives next to the data instead of scattered across every endpoint.

Q. Explain `FOR UPDATE SKIP LOCKED`.

In a `SELECT … FOR UPDATE` , Postgres locks the selected rows so no one else can modify them until your transaction ends. Normally a second transaction wanting the same rows would *block*. `SKIP LOCKED` tells it to skip already-locked rows and take

the next available ones instead. That turns a table into a concurrent-safe work queue: many workers each grab different rows with no blocking and no double-processing.

Q. Serverless vs. a persistent server — when do you pick which?

Serverless (Lambda, Vercel functions) is great for short, bursty, stateless request/response work — it scales to zero and you pay per invocation. A persistent server wins when you need long-running background work, in-memory state between requests, or long-lived connections (SSE, WebSockets), because serverless functions are killed when the response returns. SoundScript needs minutes-long jobs and open SSE streams, so persistent is mandatory.

Q. What is exponential backoff with jitter, and why the jitter?

Exponential backoff waits progressively longer between retries (1s, 2s, 4s…) so you stop hammering a struggling service. Jitter adds randomness to each wait. Without it, many clients that failed at the same instant retry at the same instant — a "thundering herd" that recreates the overload. Randomizing spreads the retries out so the service can recover.

Q. SSE vs. WebSockets vs. long polling?

Long polling repeatedly asks "anything new?" — simple but wasteful. SSE is a one-way server→client stream over a single long-lived HTTP response, ideal for progress and notifications, with automatic reconnection. WebSockets are full-duplex (both directions), needed for chat, games, or collaborative editing. Pick the least powerful tool that fits: for one-way progress, SSE.

Q. What does content-addressed storage mean?

You name a stored object by a hash of its *content* rather than an arbitrary id. Identical content produces the same key, so you get automatic deduplication and a free cache — asking for the same thing twice hits the same entry. Git works this way. SoundScript keys audio by a hash of provider+voice+model+text.

Q. Why does the anon/public API key being in the browser not compromise security?

Because it grants no privileges by itself — it only lets the client *attempt* operations, which RLS then authorizes per row against the authenticated user's JWT. The key is designed to be public. The sensitive credential is the service-role key, which bypasses RLS, and it never leaves the server.

Q. CommonJS vs. ES Modules?

CommonJS (`require` / `module.exports`) is Node's original synchronous module system. ES Modules (`import` / `export`) are the standardized, statically-analyzable system used across modern JS and the browser, supporting async loading and tree-shaking. This backend uses CommonJS for simplicity and broad compatibility; the Vite frontend uses ESM.

Q. What is CORS actually protecting against?

CORS is a browser mechanism that controls which origins may read responses from your API. It doesn't protect the server — any non-browser client ignores it — it protects *users* from a malicious site making authenticated cross-origin requests with their credentials. Because SoundScript uses Bearer tokens rather than cookies, a hostile origin can't ride along on an existing session anyway, but the allow-list is still good hygiene.

Q. What's the difference between authentication and authorization?

Authentication is proving *who* you are (the JWT verification). Authorization is deciding *what* you're allowed to do (RLS policies deciding which rows you can touch). SoundScript separates them cleanly: Supabase Auth handles the first, Postgres RLS handles the second.

Q. Why `npm ci` over `npm install` in CI/CD?

`npm ci` installs strictly from `package-lock.json` and errors if `package.json` and the lockfile disagree, giving reproducible, deterministic builds. `npm install` can update the lockfile and resolve slightly different versions. For a deploy you want the exact versions you tested, so `ci`.

Q. What is idempotency and where does it appear here?

An idempotent operation has the same effect whether you do it once or many times. It matters for retries: re-processing a document is safe because cached chunks are skipped and job/chunk claims are atomic, so a duplicate trigger doesn't double-charge or double-produce. Designing for idempotency is what makes "just retry it" a safe recovery strategy.

22 · One-minute cheat sheet

Question	One-liner
What is it?	Documents → narrated audiobooks with chapters, read-along, and a private podcast feed.
Hardest engineering?	Turning messy PDFs into prose, and orchestrating slow, paid, rate-limited synthesis safely.
The queue?	The `chunks` table, drained via `FOR UPDATE SKIP LOCKED`. No Redis.
Why persistent, not serverless?	Minutes-long jobs + open SSE streams die when a serverless function returns.
Security model?	JWT for identity, Postgres RLS for authorization — the DB enforces it, not the app.
Cost control?	Content-addressed cache + honest estimate (same code as the real upload) + daily quota.
Providers?	ElevenLabs primary, OpenAI fallback, behind a one-file abstraction with jittered retry.
Deploy?	One `render.yaml` → persistent API + static web; GitHub Actions runs tests/lint/format.
Guests?	Anonymous UUID = a real user, so one code path; upgrade keeps the same id + history.
Biggest limit?	Per-process SSE + in-process synthesis; scaling means a shared channel + the standalone worker.